

MMFileTransfer

Understanding the System — Core Flows

A visual walkthrough of the v1 implementation plan
(Django 5 + nginx + MinIO + PostgreSQL + Redis/Celery)

Internal file-transfer application - Revised architecture

How to read this document

This document turns the MMFileTransfer implementation plan into a set of clear, step-by-step flows. Each section pairs a flow diagram with a short narrative so you can understand how the system behaves before writing code. It is a companion to the full plan, not a replacement — section numbers (e.g. 5.5) point back to the source. The flows reflect the revised infrastructure (nginx, MinIO, Redis/Celery); see the next page for a summary of what changed.

Diagram legend

Shape / colour	Meaning
Purple rounded box	Entry point — a user or visitor action that starts the flow
Blue rounded box	A processing step the server performs
Teal cylinder / box	MinIO object storage or a presigned-URL step
Amber diamond	A decision point (branch on a condition)
Amber box	An intermediate / waiting state
Green box	A successful outcome, or a Celery worker step
Red box / cylinder	A rejection / failure path, or Redis
Grey cylinder	A datastore (PostgreSQL)
Dashed arrow	An alternate, conditional, async, or retry path

What the product is

MMFileTransfer is an internal file-transfer application. Authenticated users upload files (small or very large, with resumable chunked transfer), assign them to specific recipients, and optionally publish them via public token links. Administrators manage users, roles, groups, categories, retention, and email notifications. Storage is a local MinIO object store; authentication supports local accounts plus LDAP with just-in-time provisioning; and the build is organised into phases 0-7.

Architecture revision — what changed

This revision keeps every product behaviour from the original plan but swaps several infrastructure components. The flows below already reflect these changes.

Area	Original plan	This revision
Reverse proxy	Apache (mod_proxy, optional mod_xsendfile)	nginx (TLS termination, reverse proxy to Gunicorn)
Application	Django 5 server-rendered UI + Gunicorn	Unchanged — Django renders both UI and API, served by Gunicorn
File storage	Local disk under MEDIA_ROOT (files/{year}/{month}/)	MinIO (local), S3-compatible object store; same key layout as object keys
File delivery	Django FileResponse (optional mod_xsendfile)	Django authorizes, then issues a short-lived presigned MinIO URL; browser downloads direct
Metadata DB	PostgreSQL	Unchanged — PostgreSQL
Async / queue	DB-backed NotificationJob drained by OS cron; purge via OS cron	Redis broker + Celery workers; Celery Beat replaces OS cron
Transactional mail	Synchronous in-request	Unchanged — stays inline in the request (user is waiting)

Implications to keep in mind

- MinIO is S3-compatible, so Django uses an S3 client (e.g. boto3 / django-storages). Large uploads map naturally onto MinIO multipart upload; the original 50 MiB chunk threshold and SHA-256 integrity checks still apply.
- Presigned URLs mean permission and download-limit checks must complete in Django before the URL is minted; the URL itself is short-lived and carries the original filename as the download name. MinIO is never exposed unauthenticated for private files.
- Celery + Redis replace both the cron-drained notification queue and the OS cron jobs. Deferred email (assignment, welcome, expiry reminders), the retention/expiry purge, and thumbnail generation all run as Celery tasks; Celery Beat schedules the periodic ones.
- Security-critical mail (password reset, 2FA codes, public verification codes) still sends synchronously in the request — it is never handed to Celery, because the user is waiting.
- Redis is also available as Django's cache backend; the get_client_ip() trusted-proxy logic now keys on nginx's address instead of Apache's.

1. System architecture

Production traffic terminates TLS at nginx, which reverse-proxies over loopback to Gunicorn and Django. Django reads and writes metadata in PostgreSQL and stores file blobs in MinIO, a local S3-compatible object store. Background work — deferred email, the retention/expiry purge, and thumbnail generation — runs in Celery workers driven by a Redis broker, with Celery Beat scheduling the periodic jobs. Django talks to an SMTP server for email and to an LDAP directory for authentication and recipient search.

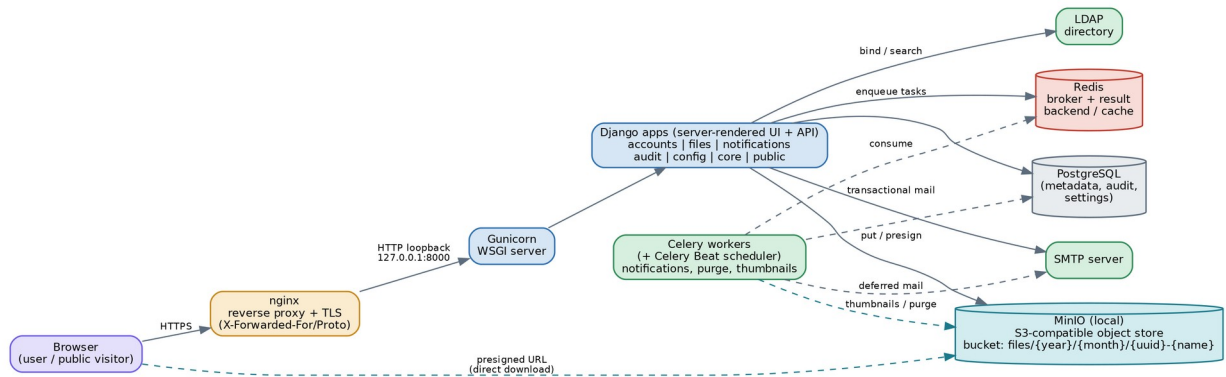


Figure - Revised request path: nginx, MinIO object storage, Redis broker, and Celery workers (3).

Key points

- Because nginx sits in front, Django never trusts `REMOTE_ADDR` directly — a single `get_client_ip()` helper reads `X-Forwarded-For` only from trusted proxy IPs (now nginx's address), so clients cannot spoof their address. Every audit/security record uses it.
- File blobs live in MinIO rather than on local disk. Django authorizes a request and then mints a short-lived presigned URL, so the browser downloads the object directly from MinIO and Django does not carry the file bytes.
- Redis is the Celery broker (and a handy Django cache backend). Celery workers run the deferred notifications, retention/expiry purge, and thumbnail tasks; Celery Beat schedules the recurring ones, replacing OS cron.
- The codebase is still split into focused Django apps: accounts (users, roles, auth, LDAP, 2FA), files (uploads, downloads, public links), notifications (email), audit (logs), config (SiteSettings singleton), core (dashboard), and an optional public app.
- `accounts.User` is a custom user model set as `AUTH_USER_MODEL` from Phase 0 — before the first migration — so all foreign keys bind to it correctly. Runtime-editable settings live in the SiteSettings singleton; deployment constants live in `settings.py` / `.env`.

2. File upload — the two-step flow

Upload is deliberately a two-step process, and the plan insists on distinct vocabulary: “complete transfer” means the bytes are written and validated; “activate file” means the metadata is saved and the file goes live. They are never both called “finalize”.

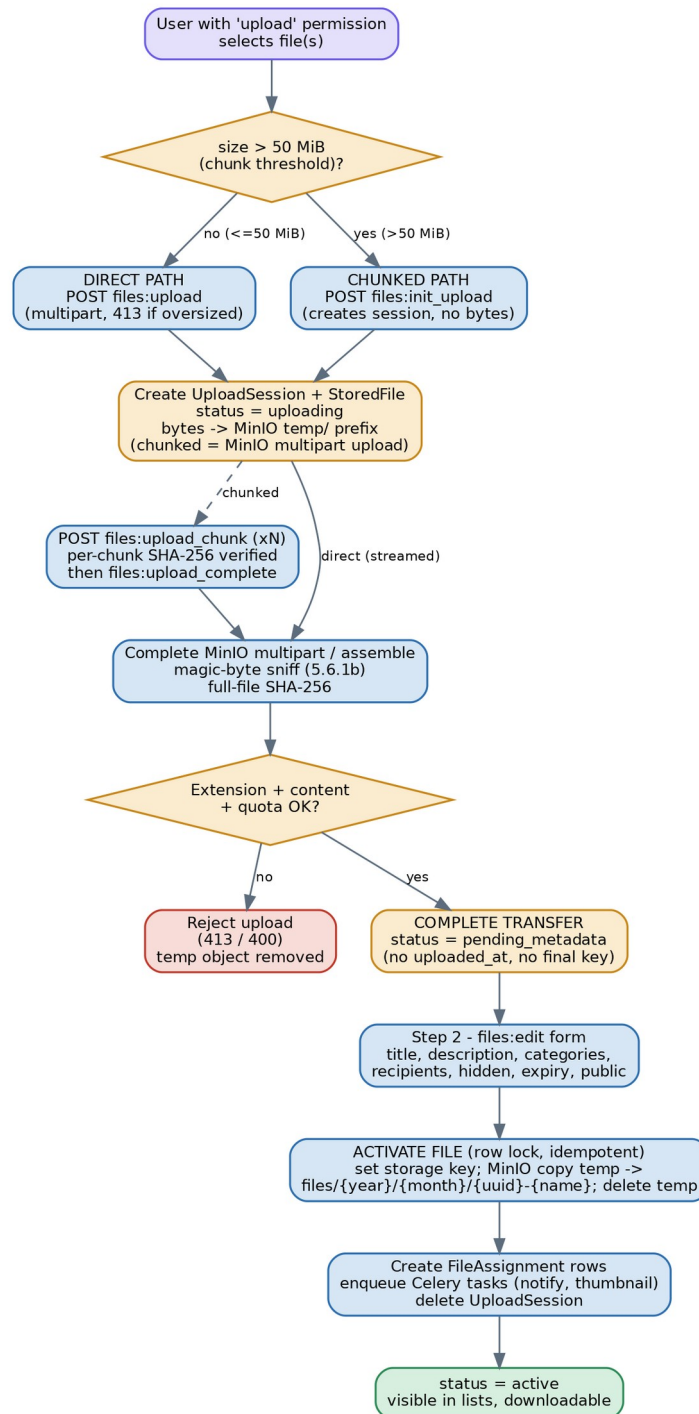


Figure - Dual-path transport into MinIO and the transition from bytes to an active file (5.5, 5.8).

Step 1 — transfer the bytes

The transport is dual-path. Files of 50 MiB or less go through a single direct multipart POST (files:upload); larger files use a resumable chunked path (files:init_upload, then repeated files:upload_chunk, then files:upload_complete), which maps onto a MinIO multipart upload. Either way the server creates an UploadSession and a linked StoredFile with status=uploading, and writes bytes only into a temporary MinIO prefix (temp/) — never the final key yet.

Integrity is checked at two layers: every chunk carries its own SHA-256 that is verified before the part is accepted, and the server computes a full-file SHA-256 as the object is assembled. A magic-byte sniff then confirms the content matches an allowed type. Once validation passes, the status becomes pending_metadata — still with no uploaded_at and no final key.

Step 2 — add metadata and activate

The same screen flow then collects title, description, categories, recipients, the hidden flag, expiry, and public options. Saving activates the file: the server sets the storage key, copies the object from the temp prefix to files/{year}/{month}/{uuid}-{name} in MinIO, sets uploaded_at, flips status to active, creates one FileAssignment row per recipient, enqueues Celery tasks (notification email and thumbnail generation), and deletes the temporary upload session and temp object. Both activate_file() and the chunk-completion step run under a row lock and are idempotent, so a retried request never double-copies objects or double-sends mail.

3. File lifecycle — status state machine

A StoredFile moves through a small, well-defined set of states. Keeping these distinct (rather than overloading a single “pending”) is what makes the upload, cleanup, and audit logic predictable.

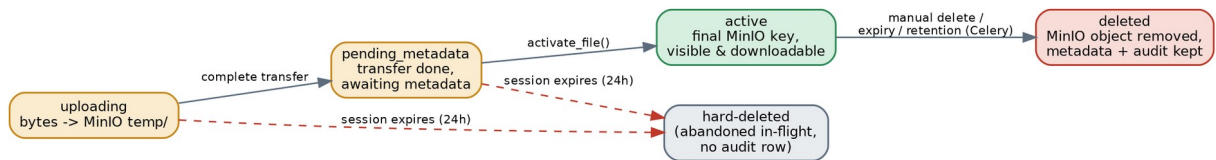


Figure - StoredFile status transitions, now backed by MinIO object keys (5.5, 5.6.2, 5.8).

Status	Meaning
uploading	Session exists; bytes are in flight to the MinIO temp prefix.
pending_metadata	Transfer complete and validated; awaiting the metadata form save.
active	Activated; on its final MinIO key; visible in lists and downloadable.
deleted	Soft-deleted — MinIO object removed, but metadata and audit history are retained.

Files that are never activated are cleaned up: the daily `purge_expired_files` task (now run by Celery Beat) hard-deletes uploading / pending_metadata rows whose upload session has expired (after ~24h) and removes their MinIO temp objects. Active files are only ever soft-deleted — by a manual delete, by per-file expiry, or by the global retention policy — so the audit trail always survives.

4. Authentication — login and two-factor

Users sign in with any of three identifiers — employee ID, email, or username — plus a password. A progressive throttle and hard block (keyed on IP and identifier) guards the endpoint before any 2FA processing happens.

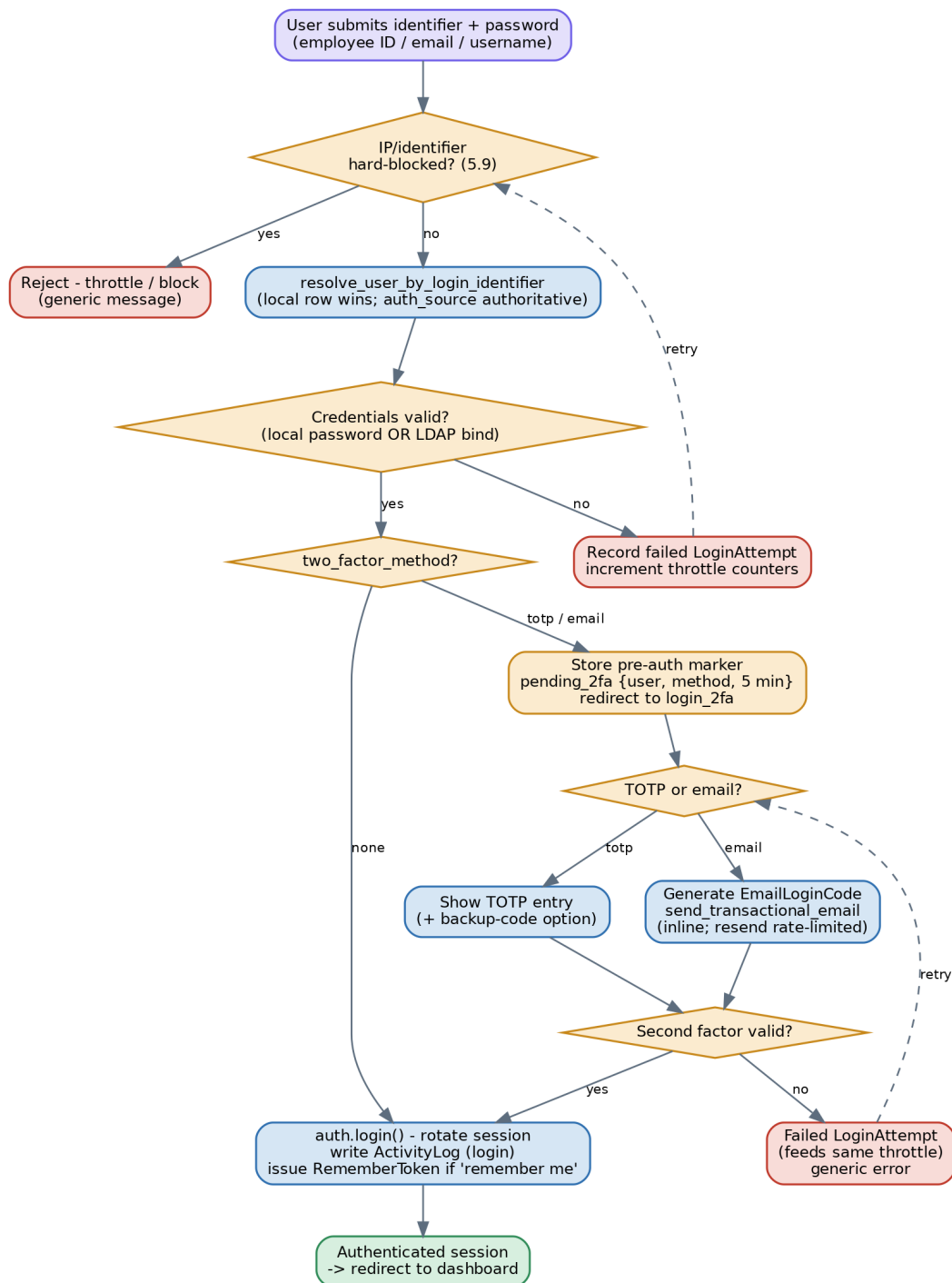


Figure - Two-step login: password first, optional second factor (5.7.1, 5.9, 5.17).

How it works

1. The request is checked against the throttle/block list first; blocked keys are rejected with a generic message.
2. `resolve_user_by_login_identifier` finds the account. A local row always wins, and `auth_source` is authoritative — there is no cross-source fallback.
3. Credentials are verified (local password or LDAP bind). Failures are recorded as `LoginAttempt` rows and feed the same throttle.
4. If the user has no second factor, the session is created immediately. Otherwise the server stores a short-lived pre-auth marker (`pending_2fa`, 5 minutes) and redirects to the 2FA step — it does NOT create an authenticated session yet.
5. For TOTP the user enters an authenticator code (or a backup code); for email the server sends a one-time code transactionally and shows the entry form.
6. On success the pre-auth marker is deleted, `auth.login()` rotates the session, an `ActivityLog` login entry is written, and only now is a remember-me token issued if requested.

Remember-me tokens are issued only after the full flow, including 2FA, so a valid remember-me login does not re-prompt for a second factor (trusted-device semantics). Those tokens are revoked on any password change/reset, 2FA enable/disable/reset, or account deactivation. Note that 2FA (TOTP and email codes) is a Phase 6 v1 security extension, not part of v1 core.

5. LDAP authentication with JIT provisioning

When no local account owns the typed identifier and LDAP is enabled, any valid directory credential may authenticate. On a user's first successful LDAP login the application creates their account automatically — “just-in-time” provisioning — with the Uploader role.

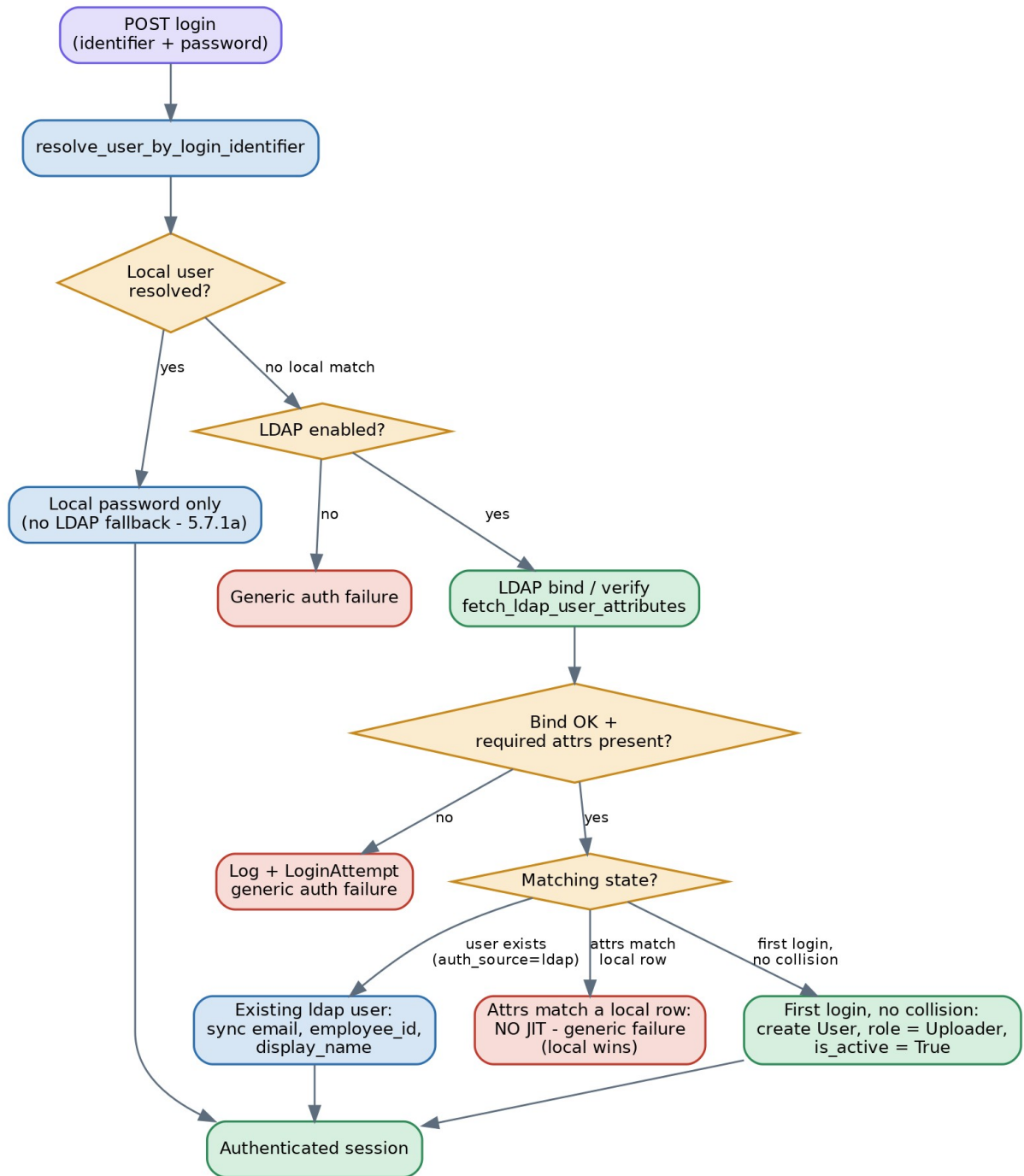


Figure - Resolution order and the local-wins collision policy (5.7.1a, 5.7.3).

The decision logic

- If a local user resolves from the identifier, only the local password is tried — there is no LDAP fallback for that account.
- Otherwise, if LDAP is enabled, the server binds against the directory and fetches the mapped attributes via a single shared `get_ldap_settings()` / `fetch_ldap_user_attributes()` path.
- If required attributes are missing, the attempt is logged and fails with a generic message.
- An existing LDAP-sourced user has their email, employee ID, and display name synced on login.
- If the fetched attributes match an existing local row, no JIT account is created — the local record wins and login fails generically.
- On a genuine first login with no collision, a new active User is created with `role=Uploader`.

LDAP configuration ships in Phase 1 (server URI, bind DN, base DN, filters, timeouts, and a Fernet-encrypted bind password). Phase 6 adds a settings UI and connection test on top of the same config source — it does not introduce LDAP for the first time.

6. Secure public download — email + verification code

A public link can optionally require email verification. When enabled, an anonymous visitor must prove control of an email address before downloading — without ever creating an account.

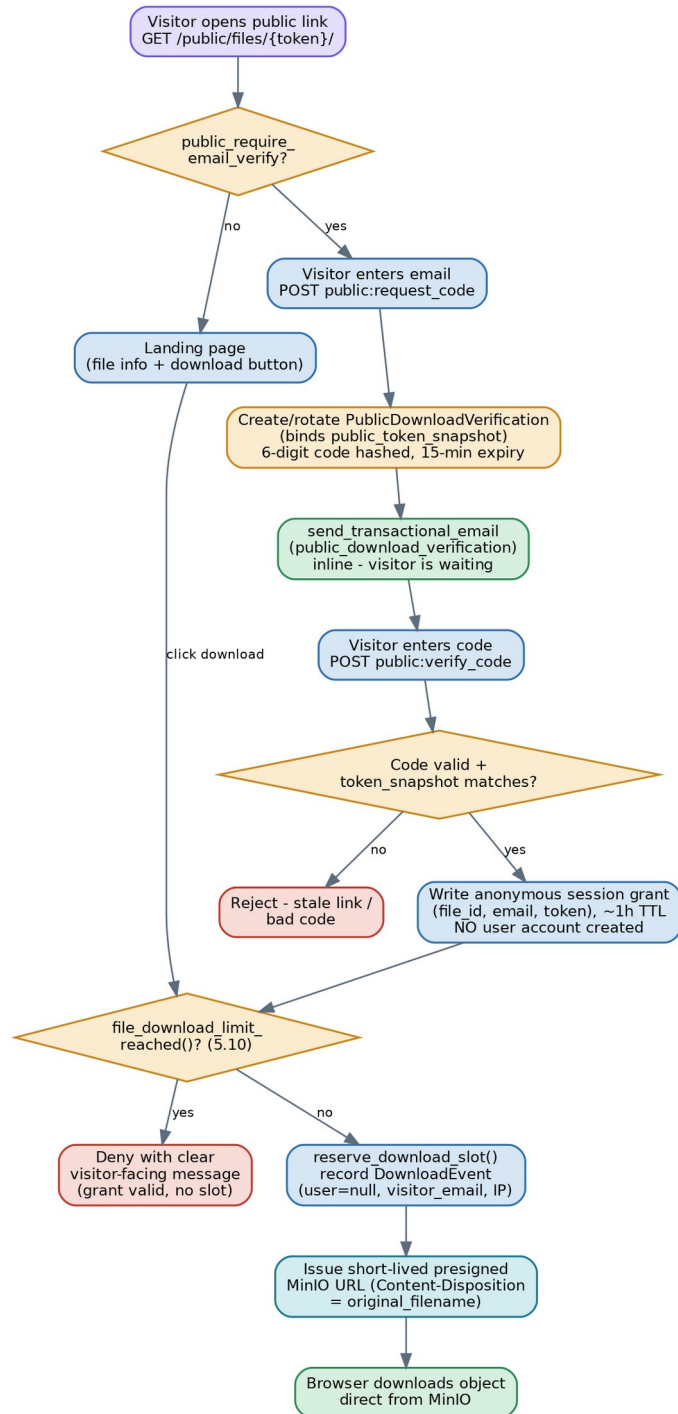


Figure - Email-verified public download, ending in a presigned MinIO URL (5.4, 5.10).

The visitor journey

7. The visitor opens `/public/files/{token}/`. If the link does not require verification, the landing page offers the download directly.
8. If verification is required, the visitor enters an email. The server creates or rotates a `PublicDownloadVerification` row, binding the challenge to the current link token, and stores a hashed 6-digit code with a 15-minute expiry.
9. The code is sent immediately via `send_transactional_email` — the visitor is waiting on the page, so it is never queued to Celery.
10. The visitor enters the code. Verification is rejected if the code is wrong or if the link's token has since changed (stale link).
11. On success the server writes an anonymous session grant scoped to `(file_id, email, token)` with a ~1-hour TTL. Only a standard session cookie is set; no user account is created.
12. Before delivery, the server re-checks the file-wide download limit and reserves a slot. If the cap was reached, it denies with a clear visitor-facing message. Otherwise it records the `DownloadEvent` and issues a short-lived presigned MinIO URL (with the original filename as the download name); the browser then downloads the object directly from MinIO.

Important edge case

Verification proves identity; it does not reserve a download slot. A visitor can verify successfully and then find the file has hit its download limit before they click — for example, a concurrent visitor took the last slot. The grant stays valid until its TTL, but the download must be denied with a clear message rather than a misleading button or a generic error.

7. Email delivery — transactional vs deferred

Email follows two separate delivery paths that share the same templates and SMTP connection but must not share the same queue. The deciding question is simple: is someone waiting on the current request for this message?

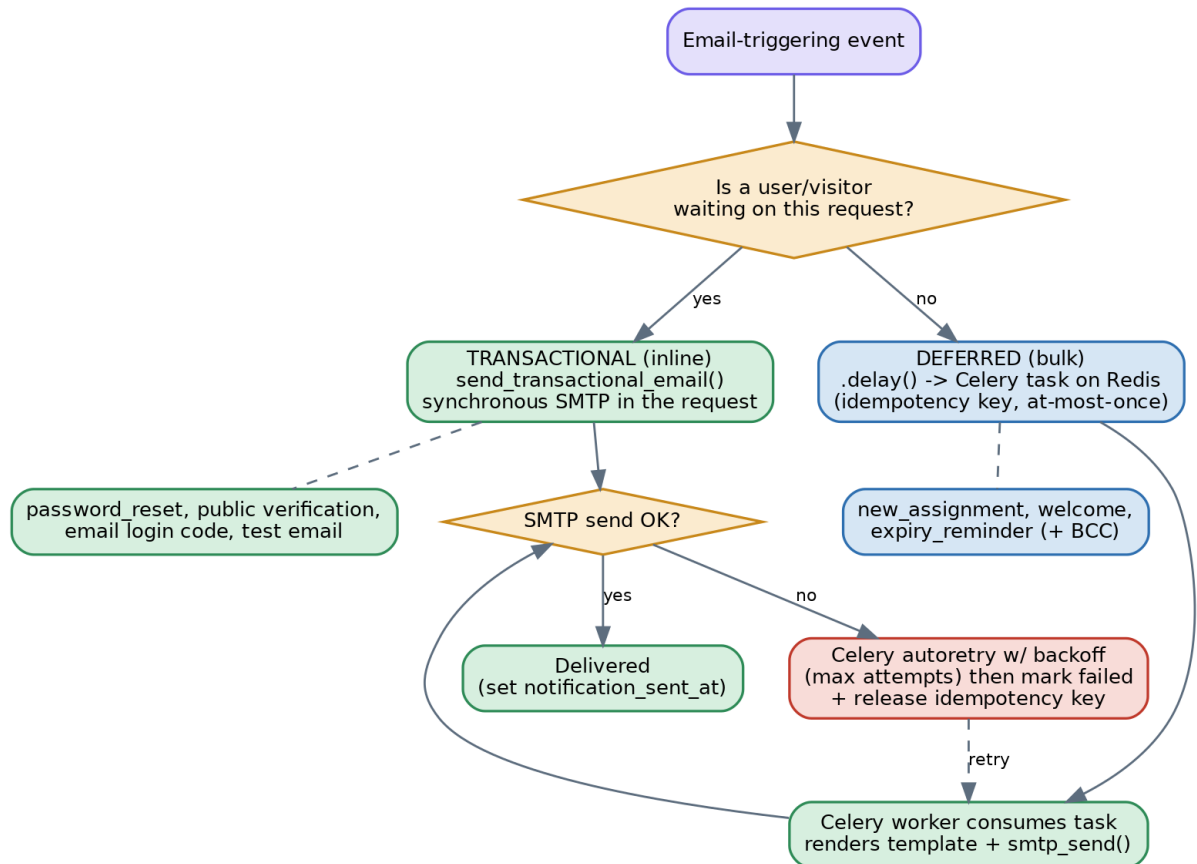


Figure - Choosing the inline path vs the Celery task queue (5.11).

Tier	When	Mechanism	Examples
Transactional	User/visitor is waiting now	<code>send_transactional_email()</code> — synchronous SMTP in the request (never Celery)	password reset, public verification code, email login code, test email
Deferred (bulk)	Fire-and-forget; seconds of delay OK	Celery task on the Redis broker, consumed by a worker	new assignment, welcome, expiry reminder (+ BCC)

Security-critical mail is never handed to Celery, because a queued task could sit briefly behind other work — unacceptable for a login code or reset link, where the user is waiting on the page. Deferred tasks use an idempotency key so the same logical notification is enqueued at most

once; Celery handles retries with backoff up to a maximum number of attempts, after which the job is marked failed and the key released so it can be re-enqueued. `notification_sent_at` is set only when SMTP delivery actually succeeds. Compared with the original plan, Celery + Redis replace the DB-backed NotificationJob table that a cron drained every few minutes.

8. Retention and expiry — the cleanup task

Two independent policies decide how long files stay in storage. Per-file expiry blocks downloads after a chosen date; global retention caps the maximum age of any stored object. Whichever condition is reached first wins, and a Celery Beat task does the physical cleanup.

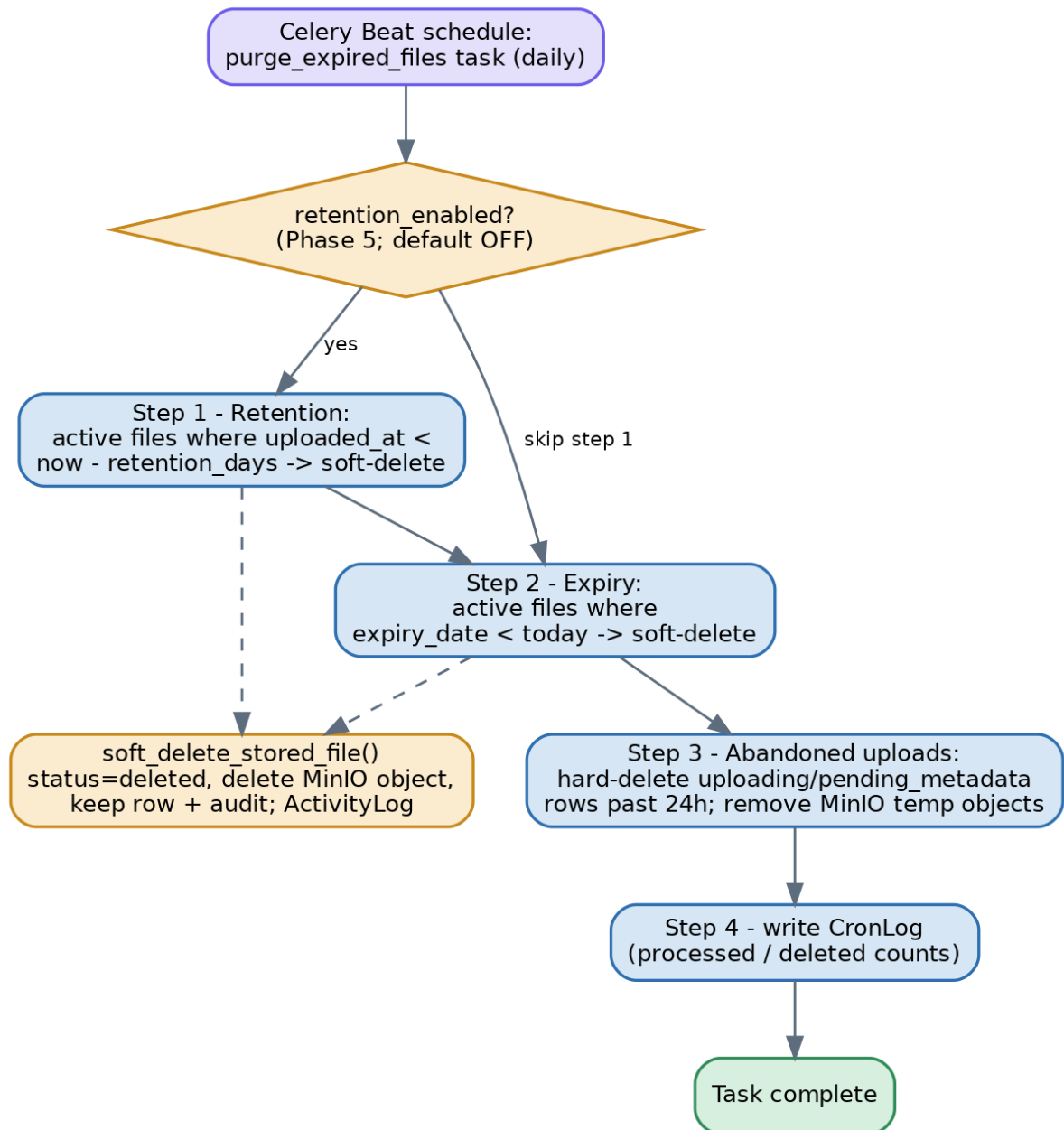


Figure - `purge_expired_files` as a Celery Beat task, with its four stable steps (5.6.2).

What the task does

13. Retention path (only if retention is enabled): active files older than `retention_days` are soft-deleted.
14. Expiry path: active files whose `expiry_date` has passed are soft-deleted, even if younger than `retention_days`.
15. Abandoned uploads: uploading / pending_metadata rows whose session expired (~24h) are hard-deleted and their MinIO temp objects removed.
16. A CronLog row records the processed and deleted counts.

Retention defaults to OFF on purpose. If it shipped on, the first scheduled run after deployment would soft-delete every file older than the default window and destroy the objects. Steps 3–4 ship in Phase 2 (so abandoned uploads are cleaned up early); the retention/expiry deletes in steps 1–2 are enabled in Phase 5, and an admin must review the settings against existing file ages before turning retention on.

9. Phased delivery at a glance

The build is sequenced into phases. v1 core is Phases 0–5; the v1 security extensions (encryption at rest and 2FA) are Phase 6; Phase 7 is deployment hardening, not new features.

Phase	Focus	Highlights
0	Project setup	Django skeleton, apps, AUTH_USER_MODEL before first migration
1	Foundation (2–3 wk)	Users/roles/permissions, multi-identifier login, LDAP config + JIT, SiteSettings, audit models, dashboard shell
2	Files core (3–4 wk)	MinIO dual-path upload + resume + SHA-256, metadata/activate, categories, my-files/my-uploads, Celery thumbnails/preview, task wiring
3	Download & public (2–3 wk)	Authenticated download via presigned URLs, ZIP, download limits, public links, secure email-verified download
4	Notifications (2 wk)	SMTP UI + templates, inline transactional + deferred Celery tasks, BCC, welcome/expiry emails
5	Admin, groups, fields (2 wk)	Dashboard/audit UI, retention enable (Celery Beat), groups, custom fields, orphan import, privacy
6	Security extensions (2–3 wk)	Encryption at rest, TOTP + email 2FA, LDAP settings UI, approval polish
7	Hardening (2 wk)	Security review, UAT, performance testing, production nginx + Gunicorn + MinIO + Redis/Celery

Estimate for a team of 2+ experienced developers: v1 core (Phases 0–5) ~ 12–14 weeks; full v1 (through Phase 6) ~ 14–18 weeks; add ~2 weeks for Phase 7. A solo developer should plan for a substantially longer calendar and use the suggested release split.